

SECURE AUTHENTICATION LAB

Introduction

In the era of Artificial Intelligence where it can also code full stack applications, learning the concept and understanding it remains a crucial skill to have. Website login pages are one of the ways adversaries may get unauthorized access to the web application. Understanding the logic on how to secure basic authentication mechanisms in a web is very important as Artificial Intelligence coding agents may disregard that feature.

It is better to prevent the incidents from happening before a SOC reports an incident and it is a true positive since there may already have been damage or losses happening. Sometimes trusting Artificial Intelligence fully to code everything end-to-end secure full stack application can be risky, since it may miss a few weak points that can be exploited, therefore there is still the need for human supervision and revision.

In the real world, this would be considered as DevSecOps work. And with the skills I am building from now, this can be useful and applied when I am working in the Security field in the future.

Methodology

In order to do this project here is the stack I will be implementing:

- Frontend: React
- Backend: Express
- Database: MySQL

I will be testing the authentication web application I created on several attack vectors, including:

- Password Strength Validation features
- SQL Injection
- Distributed Denial of Service (DDoS) attack (Rate Limiting)
- URL-Based Access Control Circumvention
- And so on

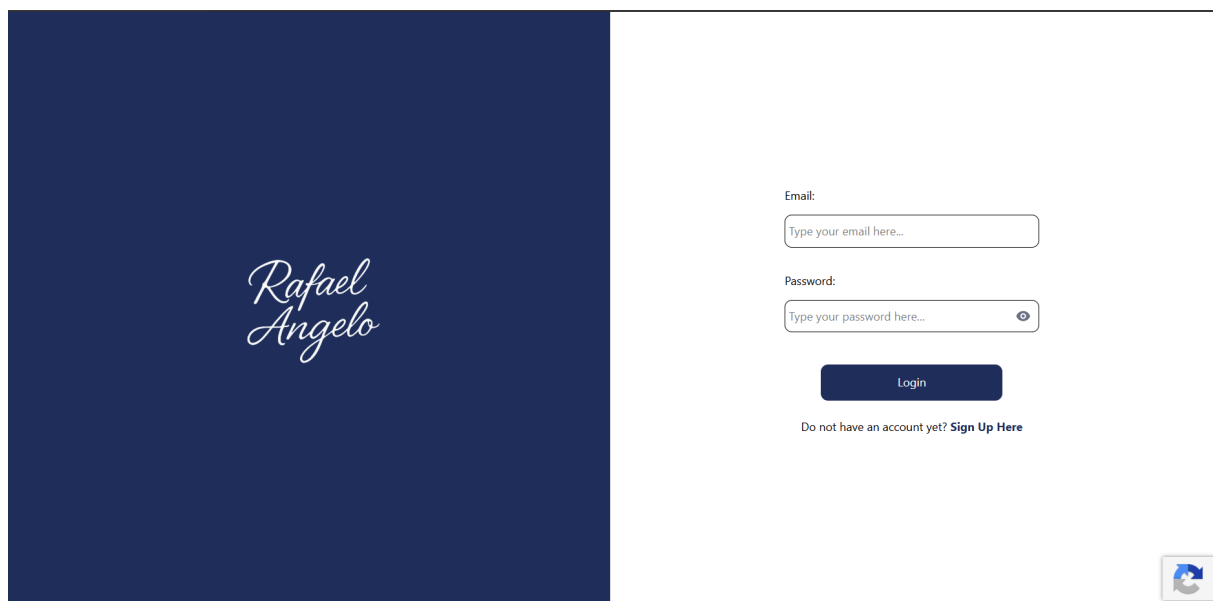
And as an additional I will also try to implement Multi-Factor Authentication by sending the email an OTP code.

Results

For the result code will be pushed at the following repository:

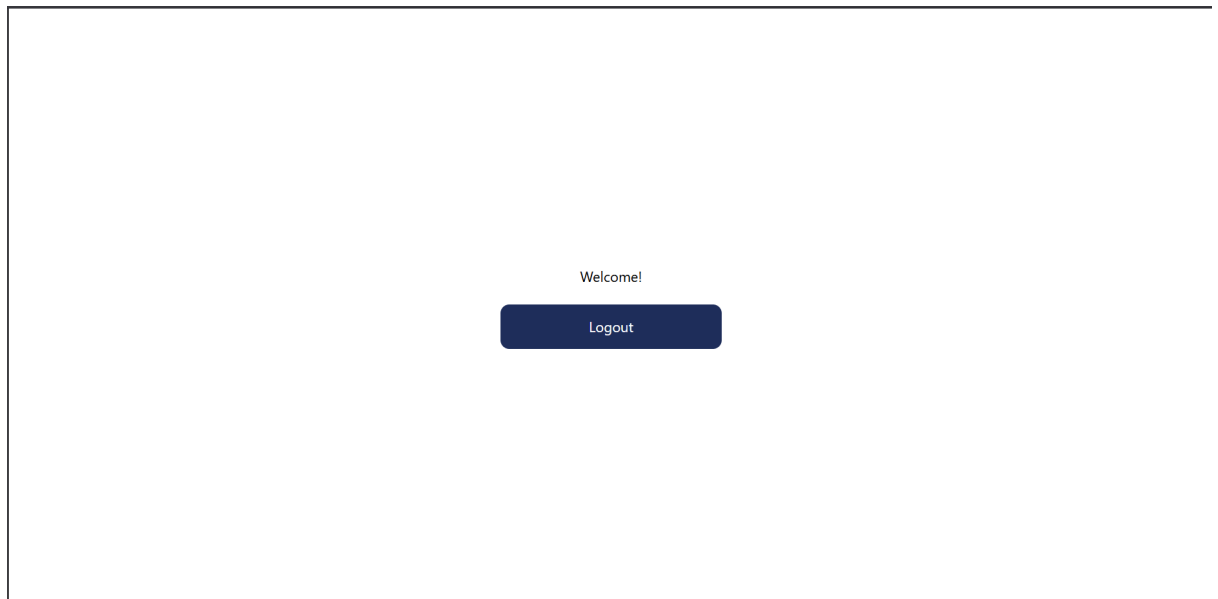
<https://github.com/RafaelAngeloChristianto/secure-auth-lab>

Upon opening the repository, there will appear 2 folders: frontend and backend serving each of their purposes based on their names.



Upon opening the link, the page will be shown like above.

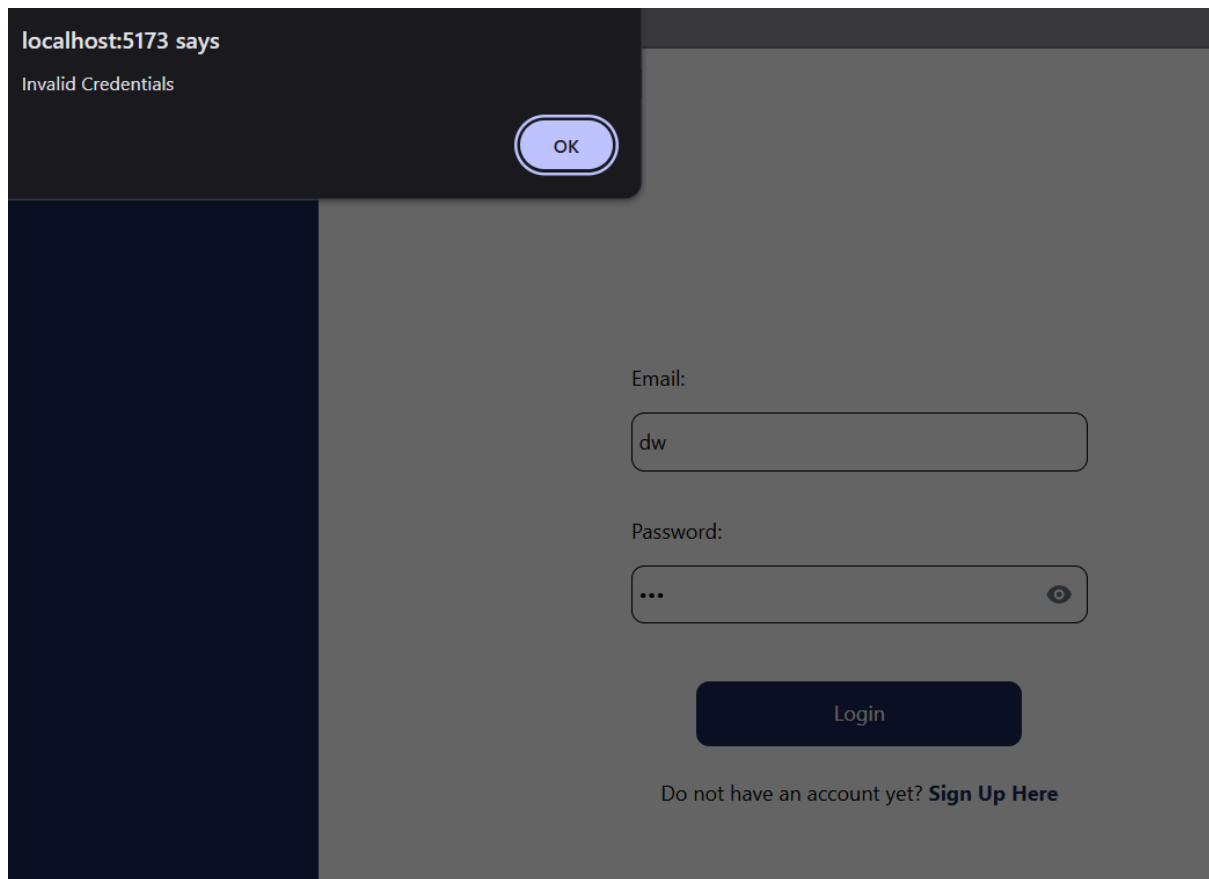
To summarize how the flow of the application works, the user signs up for an account, and their email will be inserted into a table called `otp_codes` where they contain the corresponding OTP code and expiration time of 5 minutes of each email entry. The OTP will be sent to their gmail using my private Gmail account, and it will be verified first whether it is correct or not. If it is incorrect, then there will be an alert "Invalid or Expired OTP", otherwise it will say "OTP Verified" and the user continues on to the HomeScreen and their email and hashed password.



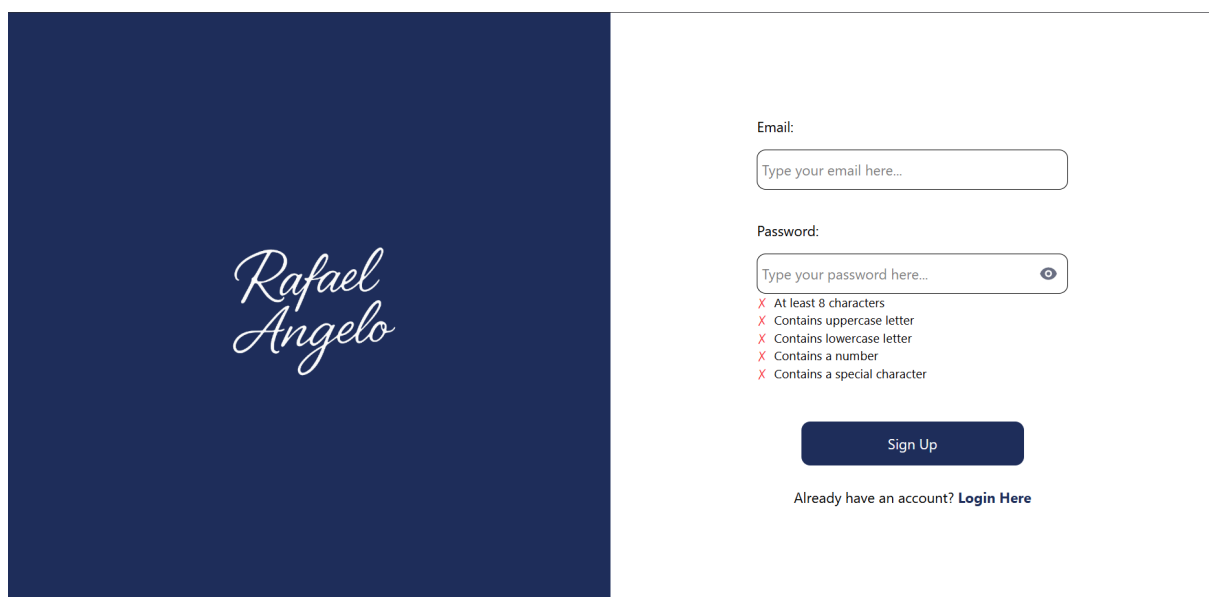
The Home Page for this website will not be anything fancy. In this case, its function is only used for users to be able to Logout, as I am focusing on the security of the Authentication system and the Home Page is just for formalities.

```
backend > .env
1  PORT = 3000
2  JWT_SECRET = 
3
4  DB_HOST=
5  DB_USER=
6  DB_PASSWORD=
7  DB_NAME=
8  DB_PORT=
9
10 AUTH_PASS = 
11
```

First to note is that the port number used, JWT secret key, SQL credentials and database information, and authentication password for the 2-way verification must be stored under the .env file and put in the .gitignore file therefore no one can see in the repository since those information still remain as confidential for the web application.



When logging in with unregistered accounts, it is better to not specify whether either the email or password is incorrect, other wise it can be brute forced by adversaries, and instead it is better to just say "Invalid Credentials" or "Account not found".



There is also the Sign Up Screen for password strength checker therefore users who sign up do not use regular and common passwords that are easily guessable and are present in common dictionaries for brute forcing such as rockyou.txt.

The screenshot shows a web application interface for signing up. At the top left, a dark overlay displays the text "localhost:5173 says" and "Password does not meet requirements:" followed by a list of requirements: "At least 8 characters", "Contains uppercase letter", "Contains a number", and "Contains a special character". An "OK" button is visible. Below this, the sign-up form includes an email input field containing "rafael@gmail.com", a password input field with masked characters ".....", and a list of requirements with status indicators: "At least 8 characters" (X), "Contains uppercase letter" (X), "Contains lowercase letter" (✓), "Contains a number" (X), and "Contains a special character" (X). A "Sign Up" button is at the bottom, and a link "Already have an account? Login Here" is at the bottom right.

If the password requirements are not met, then it will give an alert on which requirement(s) are missing.

```
app.post("/request-otp", async (req, res) => {
  const { email } = req.body;

  const otp = generateOTP();
  const expiresAt = new Date(Date.now() + 5 * 60 * 1000);

  await db.query(
    "INSERT INTO otp_codes (email, otp, expires_at) VALUES (?, ?, ?)",
    [email, otp, expiresAt],
  );

  await sendOtp(email, otp);

  res.json({ message: "OTP Sent" });
});
```

Here like shown in the screenshot above is the backend at [index.js](#), called "/request-otp" which has a task of generating the OTP and making a variable for the expiry OTP time which is 5 minutes. And then it will INSERT the email that the Sign Up Screen sent. Then, it calls the sendOtp function containing the email and otp parameters. It uses the nodemailer npm library and used the functions such as createTransport() and sendMail(). generateOTP() is a function that I created to make a 6 long integer random combination like shown below:

```
const generateOTP = () => {  
  return Math.floor(100000 + Math.random() * 900000).toString();  
};
```

sendOtp() is also a function that I created and the code will be given below:

```
const transporter = nodemailer.createTransport({  
  service: "gmail",  
  auth: {  
    user: "angelorafael0508@gmail.com",  
    pass: AUTH_PASS ,  
  },  
});  
  
const sendOtp = async (email, otp) => {  
  await transporter.sendMail({  
    from: "angelorafael0508@gmail.com",  
    to: email,  
    subject: "Your OTP Code",  
    text: `Your OTP code is ${otp}. Please do not share this with anyone.`,  
  });  
};
```

```
app.post("/verify-otp", async (req, res) => {
  const { email, otp, password } = req.body;

  if (!email || !otp || !password) {
    return res.status(400).json({ error: "Missing required fields" });
  }

  try {
    const [rows] = await db.query(
      "SELECT * FROM otp_codes WHERE email = ? AND otp = ? AND expires_at > NOW()",
      [email, otp],
    );

    if (rows.length === 0) {
      return res.status(400).json({ error: "Invalid or expired OTP" });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    try {
      await db.execute("INSERT INTO users (email, passwd) VALUES (?, ?)", [
        email,
        hashedPassword,
      ]);
    } catch (err) {
      if (err.code === "ER_DUP_ENTRY") {
        return res.status(400).json({ error: "User already exists" });
      }
      console.log("REGISTER ERROR:", err);
      return res.status(500).json({ error: "Error creating user" });
    }

    await db.query("DELETE FROM otp_codes WHERE email = ? AND otp = ?", [
      email,
      otp,
    ]);

    res.json({ message: "User created successfully" });
  } catch (err) {
    console.log("VERIFY OTP ERROR:", err);
    res.status(500).json({ error: "Server error" });
  }
});
```

Here, for the next API in the backend `"/verify-otp"` is mainly to verify whether the OTP entered is correct or not, hashing the password, registering the user and deleting the OTP entry for that email in the `otp_codes` table after the user is registered in the `users` table. It also handles duplicate entries and register errors.

```
const hashedPassword = await bcrypt.hash(password, 10);
```

As mentioned before, the users' password who sign up for this account will not be stored in plain text in the database table, however it will be hashed using bcrypt. Bcrypt is a one way hashing algorithm, and 10 in this case is the computational cost or how many times to repeat the process therefore it will be more difficult for attackers to crack it.

```
app.post("/login", async (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).send("Email and password required");
  }

  try {
    let sql = "SELECT * FROM users WHERE email = ?";
    const [rows] = await db.execute(sql, [email]);

    if (rows.length === 0) {
      return res.status(401).send("Invalid Credentials");
    }

    const users = rows[0];

    const isMatch = await bcrypt.compare(password, users.passwd);
    if (!isMatch) {
      return res.status(401).send("Invalid Credentials");
    }

    const token = jwt.sign({ id: users.id, email: users.email }, SECRET_KEY, {
      expiresIn: "1h",
    });

    res.json({ token });
  } catch (err) {
    console.log(err);
    res.status(500).send("Error logging in");
  }
});
```

This is for the login endpoint that receives the data from the frontend. It first checks for empty fields if there are any, it will give an error message. If the account is not registered, then it will send invalid credentials error. Then, there is a code for `bcrypt.compare()` and it compares the password that the user entered with the hashed password in the database, and if it is not a match, it will return

Invalid Credentials. Then next, it will generate a JWT token that expires upon 1 hour and acts as a proof that the user is logged in, each user will get a unique one and will be used for protective routes. And then last it will send it to the frontend.

```
function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;

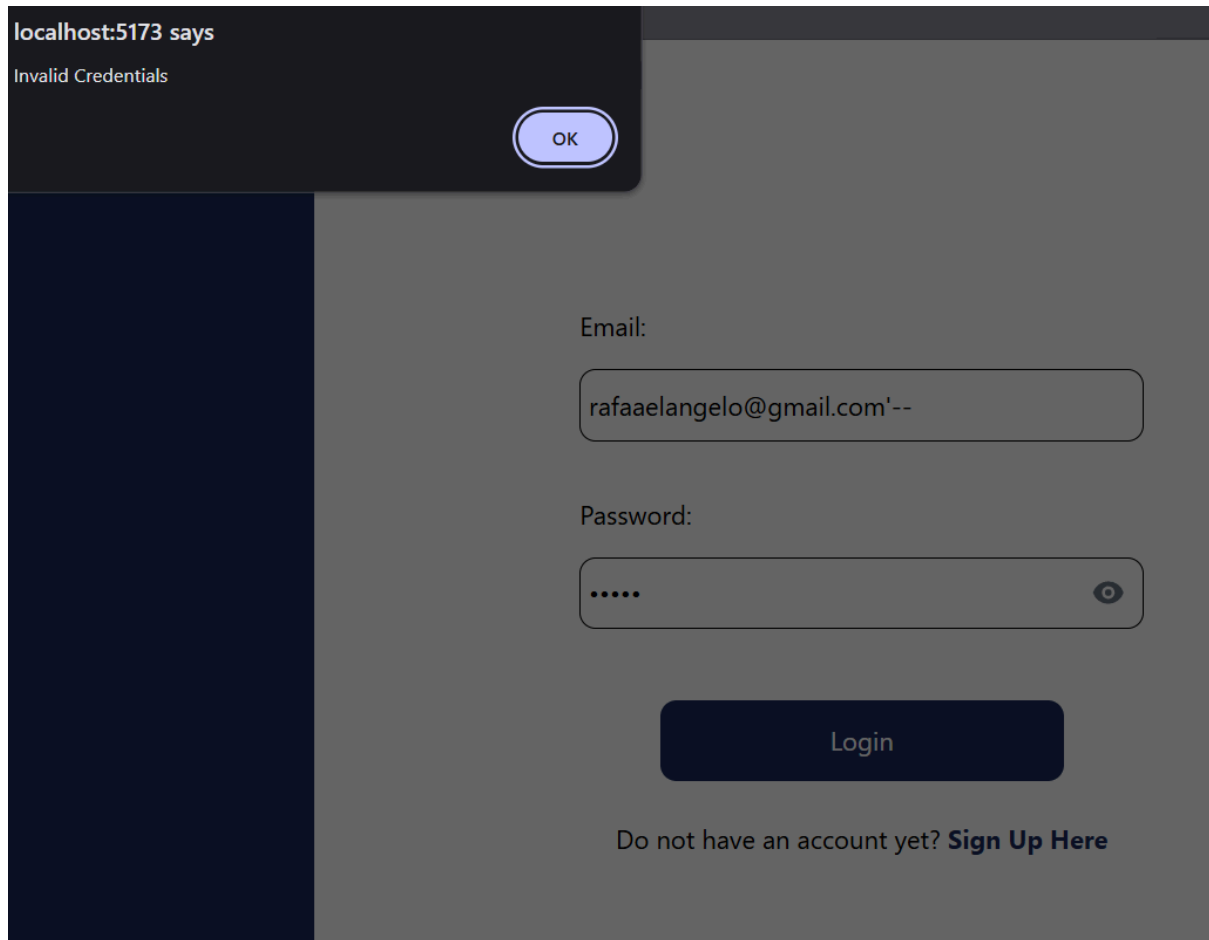
  if (!authHeader) {
    return res.status(401).send("No token");
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, SECRET_KEY);
    req.user = decoded;
    next();
  } catch (err) {
    res.status(403).send("Invalid token");
  }
}

app.get("/dashboard", authMiddleware, (req, res) => {
  res.json({ message: `Welcome to your dashboard, ${req.user.email}` });
});
```

This next one shown above is for JWT authentication token and protected routes. It gets the token created during login from the request headers. It checks if there is no token it will reject the request. Then it splits the token between spaces into a list and gets the [1] index, which is the second one. Then it will verify the token with the secret key whether it is valid and not expired. Then, next we see the "/dashboard" endpoint, and if a user requests for it, authMiddleware runs first and checks for the token, if no token then the request is rejected.



I have also tried to perform **SQL injection** attack on the authentication system that I built. However, it seems that it did not get into the SQL query. Here I will explain how I did it

```
let sql = "SELECT * FROM users WHERE email = ?";
const [rows] = await db.execute(sql, [email]);
```

Shown in the 2 lines of code above from the login endpoint, the SQL query uses "?" as a placeholder, and the email is passed separately. Therefore SQL will treat it as data instead of query. So if putting it in code and query, it will just look like *WHERE email = "rafaaelangelo@gmail.com'--"*. So it does not break the query.

```
const isMatch = await bcrypt.compare(password, users.passwd);
if (!isMatch) {
  return res.status(401).send("Invalid Credentials");
}
```

Rafael Angelo Christianto

Plus, even if somehow managed to escape out of the email query. They still need to be able to get the password hash to get through. So to summarize, checking for the email and password is really good to be separated.

For testing purposes, I will change the code to be like this, shown below:

```
try {
  let sql = `SELECT * FROM users WHERE email = '${email}' AND passwd = '${password}'`;
  const [rows] = await db.execute(sql, [email]);

  if (rows.length === 0) {
    return res.status(401).send("Invalid Credentials");
  }
  const users = rows[0];

  // const isMatch = await bcrypt.compare(password, users.passwd);
  // if (!isMatch) {
  //   return res.status(401).send("Invalid Credentials");
  // }
}
```

I will comment out the password hash check with the database and just put the variable email and password within the SQL query. And the result:

Email:	Email:
rafaaelangelo@gmail.com' OR '1'='1' --	rafaaelangelo@gmail.com' --
Password:	Password:
anything	anything
Login	Login
Do not have an account yet? Sign Up Here	Do not have an account yet? Sign Up Here

By using the 2 combinations shown above with the vulnerable code, it can bypass the login screen and straight to the Home Screen. However, I learned something too since I can fully test on authentication systems that I own and I can test on, there must be a space after the "--", otherwise if it does not it will not work.

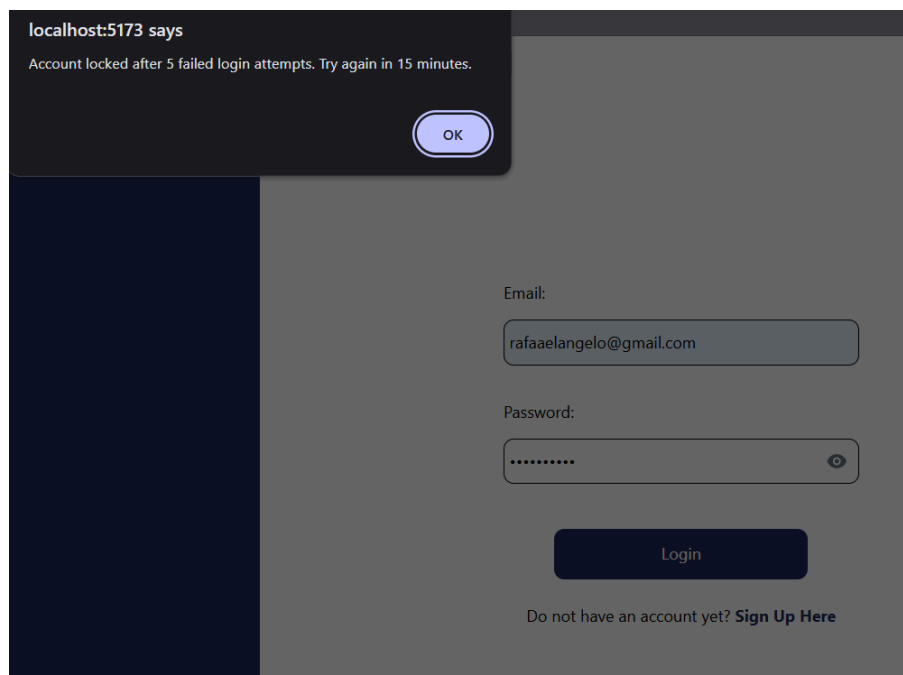
Rafael Angelo Christianito

Next, for limiting **Distributed Denial of Service (DDoS)**, I will use the npm package, called "express-rate-limit" for rate limiting.

```
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  message: "Too many login attempts. Try again later"
})
const otpLimiter = rateLimit({
  windowMs: 5 * 60 * 1000,
  max: 5,
  message: "Too many OTP requests. Try again later"
})
app.use("/login", loginLimiter)
app.use("/verify-otp", otpLimiter)
```

Here we see that there are 2 rate limits one for the login and one for the OTP.

OTP is made different since it will definitely be only 6 numbers, and it is pretty short therefore 5 is enough



The screenshot shows a web application interface. At the top left, a dark notification box displays the text "localhost:5173 says" and "Account locked after 5 failed login attempts. Try again in 15 minutes." with an "OK" button. Below this, the login form is visible. It includes an "Email:" label, a text input field containing "rafaaelangelo@gmail.com", a "Password:" label, a password input field with masked characters "....." and a toggle icon, a "Login" button, and a link "Do not have an account yet? Sign Up Here".

Here we see the implementation that after 5 login attempts, the account will be locked out to prevent brute force attacks. Below, I altered the users table in the database to add 2 columns to support the account locking feature:

- Failed_attempts is counting how many failed login attempts the user is attempting to do and when it reaches 5, the account gets locked.
- Locked_until is the time when the user's account will be unlocked which is set to be 15 minutes from the current date and time.

	id	email	passwd	created_at	failed_attempts	locked_until
▶	1	rafaaelangelo@gmail.com	\$2b\$10\$WYI0X3Ao7PHTzDDVjvJ8O8GgatAYAz...	2026-06-14 00:20:18	0	2026-06-14 00:35:40
*	NULL	NULL	NULL	NULL	NULL	NULL

```
if (failedAttempts >= 5) {
  await db.execute(
    "UPDATE users SET failed_attempts = 0, locked_until = DATE_ADD(NOW(), INTERVAL 15 MINUTE) WHERE id = ?",
    [user.id],
  );

  return res.status(403).json({
    code: "ACCOUNT_LOCKED",
    message:
      "Too many failed attempts. Your account is locked for 15 minutes.",
  });
}
```

Here in the [index.js](#) backend, there is an if condition when failed attempts gets to 5 and over, the database will execute a query that will reset the failed_attempts column to 0, and make the interval of the account locked_until to be until 15 minutes. And then it returns a status 403, with a message and code, that will be used in the frontend

```
if (data?.code === "ACCOUNT_LOCKED") {
  alert(
    "Account locked after 5 failed login attempts. Try again in 15 minutes.",
  );
}
```

Here called in the frontend mentioned that if the code is ACCOUNT_LOCKED like stated in the backend that it will give a suitable alert.

```
const handleLogin = async (e) => {
  e.preventDefault();
  console.log(email, password);

  if (!executeRecaptcha) {
    alert("Recaptcha not ready");
    return;
  }

  const token = await executeRecaptcha("login");

  try {
    const res = await axios.post("http://localhost:3000/login", {
      email: email,
      password: password,
      captchaToken: token,
    });

    if (res.data.token) {
      localStorage.setItem("token", res.data.token);
      navigate("/home");
    } else if (res.data.requireOTP) {
      navigate("/otp");
    } else {
      alert("Invalid Credentials");
    }
    console.log(res.data);

    if (res.data.token) {
      localStorage.setItem("token", res.data.token);
      navigate("/home");
    } else {
      alert("Invalid Credentials");
    }
  } catch (error) {
    if (error.response?.status === 403) {
      alert("Suspicious activity detected");
    } else {
      alert("Invalid Credentials");
    }
  }
};

const togglePasswordVisibility = () => {
  setShowPassword((prev) => !prev);
};
```

Besides that, I also have implemented CAPTCHA v3 which is the current modern standard and many modern systems have already implemented it. The v3 CAPTCHA feels better for UX since it is not seen by the users. It gives a score from 0.0 - 1.0. The higher the score means it is human, and the lower score indicates it as bots. v3 analyzes: mouse movement, typing behavior, interaction timing, browsing patterns. However, the score handling must be handled manually, such as I did in the backend shown below.

For score below 0.7, require OTP, and score below 0.5 sends that there is suspicious bot activity.

```
if (score < 0.5) {  
  return res.status(403).send("Suspicious activity");  
}  
if (score < 0.7) {  
  return res.json({ requireOTP: true });  
}
```

If the app were to be deployed, it would be best to put it behind a Cloud DDoS protection such as from Cloudflare or put a reverse proxy from nginx. And block suspicious IP if they are making too many requests, that could indicate a brute force attack.

Next, we will try to secure **URL-Based Access Control Circumvention**.

```
frontend > src > ProtectedRoute.tsx > default  
1  import { Navigate } from "react-router-dom";  
2  
3  const ProtectedRoute = ({ children }) => {  
4    const token = localStorage.getItem("token");  
5  
6    return token ? children : <Navigate to="/" replace />;  
7  };  
8  
9  export default ProtectedRoute;
```

For this project I created a new file called ProtectedRoutes.tsx like shown above and this checks whether the user has a token in local storage, and if it does not it will navigate back to the login screen.

```
<Route  
  path="/home"  
  element={  
    <ProtectedRoute>  
      <HomeScreen />  
    </ProtectedRoute>  
  }  
</Route>
```

And then shown in the screenshot above I incased the HomeScreen route using the ProtectedRoute file that I imported at App.tsx which acts as my routing file

so when the user tries <http://localhost:5173/home>, it just sends them back to the login screen.

However, this frontend only fix does not protect the site fully, since if attackers go into DevTools and enter in a fake token, the backend will still be called, therefore it is already protected by the authMiddleware function discussed previously which can let the page load but the data does not.

Next I will try to protect it from one of the primary and famous attack vectors, which is **Cross Site Scripting (XSS)**. By default React already gives some protection for XSS rather than using HTML. But, it does not secure it fully therefore we need some extra controls

So then first I installed an npm package called helmet, and it protects headers and basic XSS protection. It is just a 1 line code, excluding the variable require line.

```
app.use(helmet())
```

The code for helmet is shown above.

```
app.post("/login", async (req, res) => {
  console.log("=== LOGIN ROUTE TRIGGERED ===");
  const { email, password, captchaToken } = req.body;

  if (!email || !password) {
    return res.status(400).send("Email and password required");
  }

  if (!captchaToken) {
    return res.status(400).send("Captcha token required");
  }

  try {
    body("email").isEmail().normalizeEmail(),
    body("password").isLength({ min: 6 }),
    (req, res) => {
      console.log("Incoming login:", { email, captchaToken });
      const captchaRes = await axios.post(
        "https://www.google.com/recaptcha/api/siteverify",
        null
      );
    }
  }
});
```


Then, next I installed another npm library called express-validator and it is used to validate input and prevent any malicious input. Shown by the screenshot above it has an `isEmail()` and `normalizeEmail()` functions and also a password length of minimum of 6.

Then, I also changed how the token is stored on local storage and instead I use HTTP only cookies by changing

```
localStorage.getItem("token")
```

Into:

```
if (res.data.jwtToken) {  
  res.cookie("token", jwt, {  
    httpOnly: true,  
    secure: true,  
    sameSite: "Strict",  
  });  
}
```

This is because if it is stored in local storage, then XSS can steal it but since it is now stored in HttpOnly cookies, JavaScript cannot read it.

```
app.use(cors({  
  origin: "http://localhost:5173",  
  credentials: true  
}));  
app.use(cookieParser())
```

However, if implementing HttpOnly cookies then cors to the frontend and credentials must be activated, and also have to use `cookieParser()` which translates raw strings of cookies to readable JSON formats, since [Node.js](#) and Express do not parse cookies headers by default.